



**INSTITUTO
FEDERAL**

Santa Catarina

Câmpus
São José

Interação entre Tarefas

Sistemas Operacionais - Laboratório 7

Curso: Engenharia de Telecomunicações
Professor: Rogerio Pereira Junior

Aluno:
Victor E. de L. Guerra

08/05/2026

Sumário

1	Mutex e Semáforo em Java	2
1.1	Mutex com ReentrantLock	2
1.2	Semáforo binário com Semaphore(1)	2
1.3	Semáforo contador com Semaphore(3)	2
2	Conta bancária	2
2.1	Sem proteção — a disputa exposta	3
2.2	Solução com Mutex (ReentrantLock)	3
2.3	Solução com Semáforo	3
2.4	Solução com Monitor (synchronized)	3
3	Pizzaria com buffer de 2 pizzas	3
3.1	Cenário 1: cozinheiro rápido, entregador lento	3
3.2	Cenário 2: cozinheiro lento, entregador rápido	4
3.3	Por que notifyAll() em vez de notify()?	4
4	Limitações dos semáforos	4
4.1	Resumo das consequências	5
5	Conclusão	5

1 Mutex e Semáforo em Java

A linguagem Java oferece, no pacote `java.util.concurrent`, as classes `ReentrantLock` e `Semaphore` que correspondem diretamente ao `pthread_mutex_t` e ao `sem_t` de POSIX. A correspondência entre as primitivas é direta:

C / POSIX	Java / <code>java.util.concurrent</code>
<code>pthread_mutex_t lock;</code>	<code>ReentrantLock lock = new ReentrantLock();</code>
<code>pthread_mutex_lock(&lock);</code>	<code>lock.lock();</code>
<code>pthread_mutex_unlock(&lock);</code>	<code>lock.unlock();</code>
<code>sem_t s; sem_init(&s,0,N);</code>	<code>Semaphore s = new Semaphore(N);</code>
<code>sem_wait(&s);</code>	<code>s.acquire();</code>
<code>sem_post(&s);</code>	<code>s.release();</code>

A diferença prática é que em Java o `unlock()` (ou o `release()`) deve ficar em um bloco `finally` — assim o lock é liberado mesmo se a seção crítica lançar exceção, evitando deadlocks por vazamento.

1.1 Mutex com `ReentrantLock`

Reescrita do `m8-mutex.c`: 100 *threads* incrementam um contador 100 000 vezes cada uma. Sem proteção, várias atualizações são perdidas; com `ReentrantLock` o resultado fecha em $100\,000 \times 100 = 10\,000\,000$. O código (`MutexExemplo.java`) segue o padrão `lock.lock()` antes da seção crítica e `lock.unlock()` dentro de um `finally`.

1.2 Semáforo binário com `Semaphore(1)`

Reescrita do `m5-semaforo.c`. Com uma única “permissão” o semáforo funciona como um mutex: a primeira *thread* a chamar `s.acquire()` consome a vaga; as demais ficam bloqueadas até o `s.release()` correspondente. O programa (`SemaforoExemplo.java`) executa as mesmas 100 *threads* $\times$ 100 000 incrementos do exemplo anterior e também alcança o total esperado de 10 000 000.

1.3 Semáforo contador com `Semaphore(3)`

Reescrita do `m6-semaforo.c`. O valor inicial $N = 3$ permite no máximo três *threads* simultaneamente dentro da seção crítica — útil para limitar acesso concorrente a um recurso (banco de dados, *pool* de conexões, vagas de estacionamento). Em `BancoConexoes.java` criei o semáforo como `new Semaphore(3, true)` (modo *fair*), o que serve as *threads* em ordem de chegada e reduz *starvation* ao custo de um pouco de *throughput*. Nos testes com 10 *threads*, nunca houve mais que 3 conectadas ao mesmo tempo.

2 Conta bancária

O `conta.c` original tem uma condição de disputa: a função `depositar` faz três passos (ler, somar, escrever) com um `usleep(100)` entre eles, forçando a troca de contexto. Em Java reproduzi a mesma ideia com `Thread.sleep(0, 100_000)` (100 μ s de pausa entre a leitura e a escrita do saldo). A seguir comparo a versão sem proteção contra as três soluções pedidas.

2.1 Sem proteção — a disputa exposta

`ContaProblema.java` reproduz o cenário do `conta.c`: duas *threads* depositam simultaneamente em uma mesma variável `saldo`, sem nenhum mecanismo de sincronização. Como `t1` lê `saldo=100`, dorme, e nesse intervalo `t2` também lê 100 e grava 130, quando `t1` retoma sobrescreve com $100 + 50 = 150$. O depósito de `t2` é perdido. Em todas as 5 execuções de teste o saldo final ficou em 130, evidenciando a perda do depósito de 50. O problema é determinístico aqui porque o `sleep` força a interleaving.

2.2 Solução com Mutex (`ReentrantLock`)

`ContaMutex.java` envolve as três operações da função depositar (leitura, soma, escrita) entre `lock.lock()` e `lock.unlock()` (em `finally`). Com isso, mesmo que o `sleep` continue forçando a troca de contexto, a outra *thread* fica bloqueada na entrada do método e o saldo final é sempre 180.

2.3 Solução com Semáforo

`ContaSemaforo.java` usa um `Semaphore(1)`, equivalente funcional ao mutex. `s.acquire()` antes da seção crítica e `s.release()` dentro de um `finally`. Resultado idêntico ao do `ReentrantLock`: 180 em todas as execuções.

2.4 Solução com Monitor (`synchronized`)

Aqui aplico a dica do enunciado: `Thread.sleep(0)` dentro do método `synchronized`. Em `ContaMonitor.java` a conta é encapsulada em uma classe interna e o método depositar é `synchronized`. Como o lock do monitor **não** é liberado durante o `sleep` (diferente de `wait()`), a outra *thread* fica bloqueada na entrada do método e o resultado segue correto (180).

3 Pizzaria com buffer de 2 pizzas

A Mesa original (`Mesa.java`) só comporta uma pizza por vez — enquanto o cozinheiro não retira a anterior, ele bloqueia. Para permitir que o cozinheiro “estoque” até duas pizzas, transformei a mesa em um buffer com fila usando `Queue<String>` e capacidade `CAP=2`, no arquivo `MesaBuffer.java`. Os métodos pedidos pelo enunciado são todos usados:

- `fila.add(pizza)` — cozinheiro adiciona uma pizza nova;
- `fila.size()` — usado para checar se atingiu a capacidade máxima;
- `fila.isEmpty()` — entregador checa se há algo a retirar;
- `fila.poll()` — entregador retira a pizza mais antiga (FIFO).

A sincronização passou a usar `notifyAll()` no lugar de `notify()`, garantindo que qualquer *thread* bloqueada em `wait()` (cozinheiro esperando vaga, ou entregador esperando pizza) seja acordada quando o estado muda.

O driver `PizzariaBuffer.java` aceita a flag `-DcozinheiroLento=true|false`, que troca os tempos de `Thread.sleep` para testar os dois extremos.

3.1 Cenário 1: cozinheiro rápido, entregador lento

Tempo de preparo = 200 ms; tempo de entrega = 1500 ms. A fila enche até 2/2 e o cozinheiro precisa esperar o entregador retirar uma pizza — mensagens “mesa cheia (2/2)” aparecem repetidamente no log.

3.2 Cenário 2: cozinheiro lento, entregador rápido

Tempo de preparo = 1500 ms; tempo de entrega = 200 ms. A fila quase sempre fica vazia: o entregador retira a única pizza e fica em `wait()` até o cozinheiro produzir a próxima — mensagens “mesa vazia” aparecem entre cada pizza.

3.3 Por que `notifyAll()` em vez de `notify()`?

Com um único produtor e um único consumidor, `notify()` bastaria. Mas o exemplo é facilmente generalizável para vários cozinheiros e entregadores, e nesse caso `notify()` pode acordar a *thread* “errada” (ex.: acordar outro produtor quando quem estava esperando era um consumidor), causando deadlock. `notifyAll()` acorda todas as *threads* bloqueadas no monitor; o `while` em volta do `wait()` cuida de checar a condição novamente após o *wake-up*.

4 Limitações dos semáforos

Apesar de teoricamente suficientes para qualquer problema de sincronização, semáforos são primitivas de baixo nível com armadilhas conhecidas. A seguir listo as principais limitações encontradas em Tanenbaum (*Modern Operating Systems*) e Silberschatz (*Operating System Concepts*), e o que cada uma pode provocar.

1. **Não há associação explícita entre semáforo e recurso.** Um semáforo é apenas um inteiro com `wait/signal`. O compilador não verifica se o programador adquiriu o semáforo antes de tocar no dado protegido. Foi exatamente essa limitação que motivou o conceito de Monitor (Hoare, 1974), que encapsula dado e seção crítica.
2. **Inversão / esquecimento de `wait` ou `signal`.** Esquecer um `wait` expõe a região crítica (race silenciosa); esquecer um `signal` causa deadlock; trocar a ordem causa comportamento errático.
3. **Deadlock por ordem de aquisição.** Quando uma tarefa precisa de mais de um semáforo, a ordem importa. O jantar dos filósofos é o exemplo clássico: cada um pega o garfo da esquerda e tenta o da direita; todos travam.
4. **Starvation.** Sem fila justa (FIFO), uma *thread* pode nunca ser acordada quando há disputa. Em Java mitiga-se com `new Semaphore(n, true)`, ao custo de *throughput*.
5. **Inversão de prioridade.** Uma *thread* de baixa prioridade que detém o semáforo bloqueia uma de alta prioridade. Caso famoso: Mars Pathfinder (1997). A solução exige protocolos como *priority inheritance*, externos ao semáforo padrão.
6. **Composição difícil.** Operações compostas (ex.: “transferir entre duas contas atomicamente”) não saem naturais com semáforos. Locks reentrantes e monitores compõem melhor.
7. **Não-reentrância.** Semáforos POSIX não são reentrantes: a mesma *thread*, ao tentar dar um segundo `wait` em um semáforo binário que já detém, trava em si mesma. (O `Semaphore` de Java segue o mesmo princípio; quem precisa de reentrância usa `ReentrantLock` ou `synchronized`.)
8. **Espera ocupada em implementações ingênuas.** Implementações didáticas em *user-space* viram `while (s==0);`, queimando CPU. Implementações reais usam suporte do SO, mas o conceito ainda demanda cuidado em código de baixo nível.
9. **Ausência de espera por condição complexa.** Semáforos sinalizam apenas “há vaga / não há vaga”. Para condições como “buffer com pelo menos 3 itens e horário antes das 18h”, o programador precisa combinar vários semáforos — propenso a erros. Variáveis de condição (`wait/notify`) e monitores resolvem com mais clareza.

4.1 Resumo das consequências

Falha de uso	Consequência
Esquecer <code>wait</code>	Race condition, dados corrompidos
Esquecer <code>signal</code>	Deadlock
Ordem inversa de locks	Deadlock circular
Sem <i>fairness</i>	Starvation
Prioridades sem protocolo	Inversão de prioridade
Recurso não encapsulado	Bug invisível para o compilador

5 Conclusão

A reescrita em Java mostrou que `ReentrantLock` e `Semaphore` oferecem o mesmo poder das primitivas POSIX usadas em C, com a vantagem do `try/finally` (que evita deadlocks por exceção). No problema da conta bancária, os três mecanismos — mutex, semáforo e monitor — eliminaram a condição de disputa e produziram saldo final 180 em todas as execuções, sendo o monitor a opção mais idiomática em Java.

A versão da pizzeria com buffer evidencia a vantagem do produtor/consumidor desacoplado por uma fila: o cozinheiro não fica bloqueado em cada pizza, podendo estocar até duas. Os dois cenários de teste (*cozinheiro lento* vs. *rápido*) deixaram claro o comportamento esperado de cada lado bloqueando quando o buffer fica vazio ou cheio.

Por fim, ainda que semáforos resolvam, em teoria, qualquer problema de sincronização, suas limitações práticas (deadlock, starvation, inversão de prioridade, falta de encapsulamento) explicam por que as linguagens modernas oferecem mecanismos de mais alto nível como `synchronized`, `BlockingQueue` e `ReentrantLock`.